СЕМИНАРСКИ РАД

Дизајн и анализа алгоритама

Примена проширеног Дајкстриног алгоритма за рутирање под променљивим оптерећењем рачунарске мреже

Студент: Ивана Ђурић

Број индекса: 274/2022

Професор: Филип Марић

Београд, 2026.

1. Увод

Рачунарске мреже представљају скуп међусобно повезаних рачунара, рутера и других мрежних уређаја који омогућавају размену података. Један од основних проблема у функционисању рачунарских мрежа јесте одређивање путање којом ће подаци бити пренети од извора до одредишта.

Када корисник приступа некој интернет страници, шаље поруку, обавља видео-позив или преузима датотеку, подаци се не преносе нужно директно од његовог уређаја до одредишног сервера. Подаци се деле на пакете који пролазе кроз већи број мрежних уређаја. Сваки рутер на путу одређује коме ће пакет бити прослеђен. У већим рачунарским мрежама између извора и одредишта може постојати више различитих путања.

Избор путање је важан јер различите путање могу имати различите карактеристике. Неке могу имати мање кашњење, док друге могу пролазити кроз тренутно оптерећеније мрежне везе. Поред тога, стање рачунарске мреже није константно. Број корисника и количина података који се преносе мењају се током времена, па се мења и оптерећење појединих делова мреже.

Због тога је проблем проналажења оптималне путање повезан са алгоритмима над графовима. Рачунарска мрежа може се представити као граф у коме чворови представљају рутере, а гране представљају мрежне везе између њих. Тежинама грана могу се придружити различити параметри, као што су кашњење или оптерећење.

У овом раду разматра се примена Дајкстриног алгоритма и његово проширење за случај када се при избору путање, поред кашњења мрежних веза, узима у обзир и њихово тренутно оптерећење. Циљ је направити симулацију у којој се променом оптерећења мрежних веза може мењати и изабрана оптимална путања.

2. Рутирање података у рачунарским мрежама

Рутирање представља процес одређивања путање којом ће подаци бити прослеђени кроз рачунарску мрежу од извора до одредишта. У том процесу важну улогу имају рутери. Рутер прима мрежни пакет, анализира информације потребне за његово прослеђивање и одређује следећи уређај коме пакет треба послати.

Мрежу можемо представити помоћу графа:

- чворови представљају рутере,
- гране представљају директне мрежне везе између рутера,
- тежине грана представљају одређене карактеристике мрежних веза.

На пример, једноставна мрежа може изгледати овако:

$$R1 \rightarrow R2 \rightarrow R4$$

или:

$$R1 \to R3 \to R4$$

У овом случају постоје најмање две могуће путање од рутера $R1$ до рутера $R4$. Ако свака веза има различито кашњење и оптерећење, ове две путање нису нужно једнако повољне.

На пример, путања $R1 \to R2 \to R4$ може имати мање кашњење, али ако је веза $R1$–$R2$ веома оптерећена, друга путања може представљати повољнији избор према критеријуму који користи алгоритам.

Овакав проблем може се посматрати као проблем проналажења оптималног пута у графу.

3. Кашњење у рачунарској мрежи

Кашњење представља време које је потребно да подаци, односно мрежни пакет, стигну од једне тачке у рачунарској мрежи до друге. На кашњење утичу различити фактори, као што су физичка удаљеност између уређаја, карактеристике мрежне везе, број уређаја кроз које пакет пролази, као и тренутно стање мреже.

У контексту рутирања, кашњење се може посматрати као једна од тежина грана графа који представља рачунарску мрежу. Ако је, на пример, кашњење на вези између рутера $R1$ и $R2$ једнако 5 ms, то значи да је за пренос података преко те везе потребно приближно 5 милисекунди.

Уколико се подаци преносе преко више мрежних веза, укупно кашњење представља збир кашњења на свим везама које припадају изабраној путањи.

На пример, за путању

$$R1 \to R2 \to R4$$

ако је кашњење на вези $R1$–$R2$ једнако 5 ms, а на вези $R2$–$R4$ такође 5 ms, укупно кашњење је:

$$5 + 5 = 10 \text{ ms}.$$

Слично, ако путања садржи више веза, њихова појединачна кашњења се сабирају. На пример, за путању

$$R1 \to R2 \to R4 \to R5$$

са кашњењима од 5 ms, 5 ms и 6 ms, укупно кашњење износи:

$$5 + 5 + 6 = 16 \text{ ms}.$$

Мање кашњење обично значи да ће подаци брже стићи до одредишта. Због тога је кашњење један од важних параметара приликом избора путање кроз рачунарску мрежу. Међутим, само посматрање кашњења није увек довољно, јер се оптерећење мрежних веза може мењати током времена. Због тога се у овом раду, поред кашњења, узима у обзир и тренутно оптерећење мрежних веза.

4. Оптерећење мрежних веза

Оптерећење мрежне везе представља количину података која се тренутно преноси преко те везе у односу на њен укупни капацитет. На пример, ако мрежна веза има капацитет од 100 Mbps, а тренутно се преко ње преносе 20 Mbps података, њено оптерећење износи 20%.

Оптерећење мрежних веза није константно и може се мењати током времена. На њега утиче број корисника који користе мрежу, количина података која се преноси, као и врста мрежног саобраћаја. На пример, током периода када велики број корисника истовремено приступа интернету, одређене мрежне везе могу постати значајно оптерећене.

Велико оптерећење може довести до повећања времена потребног за пренос података. Због тога путања која има мало основно кашњење не мора увек бити најповољнија. Ако је нека од њених веза тренутно веома оптерећена, друга путања са нешто већим основним кашњењем може бити погоднији избор.

У овом раду оптерећење се изражава у процентима и користи се приликом израчунавања тежине мрежне везе. За везу e тежина се одређује формулом:

$$w(e) = d(e) \cdot \left(1 + \frac{L(e)}{100}\right)$$

где је $d(e)$ основно кашњење мрежне везе, а $L(e)$ њено тренутно оптерећење изражено у процентима.

На пример, ако је основно кашњење неке везе 5 ms, а њено оптерећење 20%, тежина те везе износи:

$$w(e) = 5 \cdot \left(1 + \frac{20}{100}\right)$$

$$w(e) = 5 \cdot 1.2 = 6$$

Са друге стране, ако је иста веза оптерећена 80%, добија се:

$$w(e) = 5 \cdot \left(1 + \frac{80}{100}\right)$$

$$w(e) = 5 \cdot 1.8 = 9$$

На овај начин веће оптерећење доводи до повећања тежине мрежне везе. Проширени Дајкстрин алгоритам користи овако израчунате тежине приликом избора путање, чиме се омогућава да се при избору узме у обзир и тренутно стање мрежних веза.

5. Дајкстрин алгоритам

Дајкстрин алгоритам је један од познатих алгоритама за проналажење најкраћег пута у пондерисаном графу. Користи се за одређивање најкраће путање од једног почетног чвора до осталих чворова у графу. Услов за његову примену је да тежине свих грана буду ненегативне.

У контексту рачунарских мрежа, Дајкстрин алгоритам се може применити тако што се рутери представе као чворови графа, а мрежне везе као гране. Тежина гране може представљати, на пример, кашњење на одговарајућој мрежној вези.

Алгоритам започиње тако што се почетном чвору додељује растојање 0, док се свим осталим чворовима додељује вредност која представља бесконачност. На тај начин се означава да почетна удаљеност до осталих чворова још није позната.

Затим се бира чвор са најмањом тренутно познатом удаљеношћу. За његове суседне чворове проверава се да ли се преко изабраног чвора може пронаћи краћа путања. Ако се пронађе краћа путања, растојање до суседног чвора се ажурира.

Овај поступак се понавља све док се не обраде потребни чворови. На тај начин алгоритам постепено проналази најкраће путање од почетног чвора.

На пример, ако из рутера $R1$ до рутера $R2$ постоји веза тежине 5, алгоритам може поставити тренутно растојање до $R2$ на вредност 5. Ако се касније пронађе друга путања до $R2$ чија је укупна тежина мања, вредност се ажурира.

За ефикаснију имплементацију Дајкстриног алгоритма може се користити приоритетни ред. У програмском језику Python за ову намену може се користити модул `heapq`. Приоритетни ред омогућава да се у сваком кораку брзо изабере чвор са најмањом тренутном удаљеношћу.

У основном Дајкстрином алгоритму тежина гране представља један критеријум, на пример кашњење. У овом раду алгоритам се проширује тако што се приликом израчунавања тежине, поред кашњења, узима у обзир и тренутно оптерећење мрежне везе. На тај начин избор путање може да се прилагоди променама у стању рачунарске мреже.

6. Проширење Дајкстриног алгоритма

Основни Дајкстрин алгоритам при проналажењу најкраће путање користи тежине грана које су унапред задате и не мењају се током извршавања алгоритма. У случају рачунарских мрежа, међутим, стање мреже може да се мења. Оптерећење појединих мрежних веза није константно, па се због тога може променити и погодност одређене путање.

У овом раду Дајкстрин алгоритам се проширује тако што се тежина сваке мрежне везе израчунава

на основу њеног основног кашњења и тренутног оптерећења. На тај начин алгоритам не посматра само кашњење, већ узима у обзир и тренутно стање мрежне везе.

7. Приоритетни ред у проширеном Дајкстрином алгоритму

Приоритетни ред има важну улогу у имплементацији Дајкстриног алгоритма јер омогућава да се у сваком кораку изабере чвор који тренутно има најмању познату удаљеност од почетног чвора. На тај начин алгоритам може ефикасније да пронађе оптималну путању.

У програмском језику Python за реализацију приоритетног реда може се користити модул `heapq`. Он омогућава рад са структуром података која се назива минимална гомила (*min-heap*). Елементи се у њој организују тако да се елемент са најмањом вредношћу може добити на ефикасан начин.

На почетку алгоритма у приоритетни ред се поставља почетни чвор са растојањем 0:

$$[(0, R1)]$$

Први елемент представља тренутно познату удаљеност, док други елемент представља чвор. У овом примеру почетни чвор је $R1$ и његова удаљеност од самог себе је 0.

Када се обради неки чвор, за сваког његовог суседа израчунава се нова могућа удаљеност. Она се добија као збир тренутне удаљености до посматраног чвора и тежине везе до суседног чвора:

$$\text{нова удаљеност} = \text{тренутна удаљеност} + \text{тежина везе}$$

Ако је добијена вредност мања од до тада познате удаљености до суседног чвора, његова удаљеност се ажурира и нови пар се додаје у приоритетни ред.

На пример, ако је тренутна удаљеност до чвора $R2$ једнака 5, а тежина везе од $R2$ до $R4$ једнака 6, нова удаљеност до $R4$ износи:

$$5 + 6 = 11$$

У приоритетни ред се тада додаје пар:

$$(11, R4)$$

На овај начин приоритетни ред садржи чворове који још могу бити обрађени, заједно са њиховим тренутно познатим удаљеностима. Чвор са најмањом вредношћу има највећи приоритет.

У проширеном Дајкстрином алгоритму укупна удаљеност се израчунава коришћењем тежина које зависе од основног кашњења и тренутног оптерећења мрежних веза. Због тога промена оптерећења може довести до промене вредности у приоритетном реду, а самим тим и до избора друге путање.

Приоритетни ред се, дакле, користи за избор следећег чвора који алгоритам треба да обради, док се помоћу додатне структуре података памти претходни чвор на пронађеној путањи. На основу тих података могуће је на крају алгоритма реконструисати целу путању од почетног до одредишног чвора.

8. Одређивање оптималне путање

Циљ проширеног Дајкстриног алгоритма је да пронађе оптималну путању између изабраног почетног и одредишног рутера. Оптимална путања се одређује на основу укупне тежине свих мрежних веза које се налазе на тој путањи.

За сваку везу на путањи израчунава се тежина која зависи од основног кашњења и тренутног оптерећења. Укупна тежина путање добија се сабирањем тежина свих њених веза:

$$W(P) = \sum_{e \in P} w(e)$$

где је P путања, а $w(e)$ тежина појединачне мрежне везе.

На пример, посматрајмо две могуће путање од рутера $R1$ до рутера $R5$:

$$P_1 : R1 \rightarrow R2 \rightarrow R4 \rightarrow R5$$

и

$$P_2 : R1 \rightarrow R3 \rightarrow R4 \rightarrow R5$$

Ако је укупна тежина путање P_1 мања од укупне тежине путање P_2, проширени Дајкстрин алгоритам ће изабрати путању P_1.

Приликом рада алгоритма користи се низ односно речник растојања у коме се за сваки чвор чува тренутно најмања позната вредност. Поред тога, користи се структура у којој се памти претходни чвор за сваки чвор који је ажуриран. На тај начин се након завршетка алгоритма може реконструисати пронађена путања.

Реконструкција путање врши се кретањем уназад од одредишног чвора. На пример, ако је за одредишни чвор $R5$ претходни чвор $R4$, а за $R4$ претходни чвор $R2$, добија се:

$$R5 \leftarrow R4 \leftarrow R2 \leftarrow R1$$

Овај редослед се затим обрће како би се добила путања од почетног до одредишног чвора:

$$R1 \rightarrow R2 \rightarrow R4 \rightarrow R5$$

Важно је разликовати тежину која се користи за избор путање од укупног кашњења путање. Тежина зависи и од тренутног оптерећења мрежних веза и служи алгоритму за поређење различитих могућих путања. Након избора путање, њено укупно кашњење може се израчунати као збир основних кашњења свих веза на изабраној путањи.

На овај начин алгоритам омогућава да се приликом избора путање узме у обзир тренутно стање мреже, док се кориснику може приказати стварно укупно кашњење изабране путање.

9. Рачунање укупног кашњења

Након што проширени Дајкстрин алгоритам одреди оптималну путању, потребно је израчунати њено укупно кашњење. Укупно кашњење представља збир основних кашњења свих мрежних веза које се налазе на изабраној путањи.

Ако путања P садржи више мрежних веза, укупно кашњење се израчунава према формули:

$$D(P) = \sum_{e \in P} d(e)$$

где је $D(P)$ укупно кашњење путање, а $d(e)$ основно кашњење појединачне мрежне везе.

На пример, посматрајмо путању:

$$R1 \rightarrow R2 \rightarrow R4 \rightarrow R5$$

Ако су кашњења на овим везама:

$$R1\text{–}R2 = 5 \text{ ms}$$

$$R2\text{–}R4 = 5 \text{ ms}$$

$$R4\text{–}R5 = 6 \text{ ms}$$

укупно кашњење износи:

$$D(P) = 5 + 5 + 6 = 16 \text{ ms}$$

У овом раду важно је разликовати укупно кашњење од тежине која се користи приликом избора путање. Тежина мрежне везе се израчунава узимајући у обзир и њено тренутно оптерећење:

$$w(e) = d(e) \cdot \left(1 + \frac{L(e)}{100}\right)$$

Овако израчуната тежина користи се у проширеном Дајкстрином алгоритму како би се упоредиле различите могуће путање. Са друге стране, након што је путања изабрана, кориснику се приказује њено укупно основно кашњење.

На пример, ако је за изабрану путању укупна тежина према критеријуму алгоритма 18, то не значи да је стварно укупно кашњење 18 ms. Стварно укупно кашњење се добија сабирањем основних кашњења свих веза на изабраној путањи.

Овакав начин приказивања омогућава да се јасно раздвоје два различита појма: критеријум на основу кога алгоритам бира путању и параметар који се приказује као укупно кашњење изабране путање.

10. Имплементација апликације

За практичну реализацију описаног алгоритма направљена је апликација у програмском језику Python. Апликација представља симулацију рачунарске мреже и омогућава кориснику да изабере почетни и одредишни рутер, као и да посматра изабрану оптималну путању.

Графички кориснички интерфејс апликације реализован је коришћењем библиотеке PyQt5. За представљање и обраду графа користи се библиотека NetworkX, док се библиотека Matplotlib користи за графички приказ мреже.

Рачунарска мрежа у апликацији представљена је графом чији чворови представљају рутере, а гране представљају директне мрежне везе између њих. За сваки рутер дефинисана је ознака, на пример $R1$, $R2$, $R3$, $R4$ и $R5$.

Свакој мрежној вези придружени су подаци о основном кашњењу, капацитету и тренутном оптерећењу. На пример, једна од веза може имати следеће вредности:

- веза $R1$–$R2$,
- кашњење: 5 ms,
- капацитет: 100 Mbps,
- оптерећење: 20%.

Слично су дефинисане и остале везе у мрежи. На тај начин апликација има унапред задат пример мреже на коме се може испитивати рад алгоритма.

Приликом покретања алгоритма, за сваку мрежну везу израчунава се њена тежина према формули:

$$w(e) = d(e) \cdot \left(1 + \frac{L(e)}{100}\right)$$

Добијене тежине користе се у проширеном Дајкстрином алгоритму за проналажење оптималне путање.

Корисник у апликацији може да изабере почетни и одредишни рутер. Након покретања алгоритма, апликација проналази оптималну путању и приказује је кориснику. Поред саме путање, приказује се и укупно кашњење изабране путање.

На пример, резултат може бити приказан у облику:

$$R1 \rightarrow R2 \rightarrow R4 \rightarrow R5$$

са укупним кашњењем:

16 ms

Апликација такође омогућава промену оптерећења мрежних веза. На тај начин се може посматрати како промена стања мреже утиче на избор оптималне путање.

11. Промена оптерећења и динамичко рутирање

Једна од главних предности проширеног приступа јесте могућност прилагођавања променама у стању рачунарске мреже. Пошто се оптерећење мрежних веза временом мења, може се променити и оптимална путања између два рутера.

У апликацији корисник може да промени оптерећење одређене мрежне везе. Након промене оптерећења, тежина те везе се поново израчунава према формули:

$$w(e) = d(e) \cdot \left(1 + \frac{L(e)}{100}\right)$$

Након промене оптерећења, алгоритам се поново покреће и израчунава нове тежине свих релевантних веза. На основу тих вредности одређује се нова оптимална путања.

Уместо да се једном изабрана путања користи независно од промене стања мреже, она се може поново израчунати када се промене параметри који утичу на избор путање.

На овај начин апликација омогућава да се визуелно прикаже утицај променљивог оптерећења на процес рутирања. Корисник може да повећа или смањи оптерећење одређене везе и да затим посматра да ли је дошло до промене изабране оптималне путање.

12. Анализа сложености

Сложеност алгоритма описује количину ресурса, пре свега време и меморију, која је потребна за његово извршавање у зависности од величине улазних података. У случају алгоритама над графовима, сложеност се најчешће изражава у зависности од броја чворова и грана.

У овом раду користи се проширени Дајкстрин алгоритам са приоритетним редом реализованим помоћу структуре `heapq`. Нека је V број чворова графа, а E број његових грана.

Приликом обраде сваког чвора алгоритам разматра његове суседне чворове и проверава да ли се до њих може стићи путем мање укупне тежине. Свака мрежна веза може бити разматрана током извршавања алгоритма.

Коришћењем приоритетног реда, временска сложеност Дајкстриног алгоритма је:

$$O((V + E) \log V)$$

За повезане графове, уобичајено се може записати и као:

$$O(E \log V)$$

Проширење алгоритма које се користи у овом раду не мења ред временске сложености. Израчунавање тежине мрежне везе према формули

$$w(e) = d(e) \cdot \left(1 + \frac{L(e)}{100}\right)$$

представља операцију константне сложености, односно $O(1)$. Због тога овај додатни прорачун не мења укупну асимптотску сложеност алгоритма.

Меморијска сложеност зависи од броја чворова и грана који се чувају у структури графа, као и од додатних структура које алгоритам користи. За чување растојања и претходних чворова потребно је $O(V)$ меморије, док сам граф захтева $O(V + E)$ простора.

Према томе, меморијска сложеност алгоритма је:

$$O(V + E)$$

Оваква сложеност омогућава примену алгоритма и на веће графове, иако стварно време извршавања зависи од броја рутера, броја мрежних веза и начина имплементације.

13. Закључак

У овом раду приказана је примена проширеног Дајкстриног алгоритма на проблем рутирања података у рачунарској мрежи са променљивим оптерећењем. Рачунарска мрежа представљена је као граф у коме чворови представљају рутере, а гране представљају мрежне везе између њих.

Основни Дајкстрин алгоритам омогућава проналажење најкраће путање на основу задатих тежина грана. У овом раду алгоритам је проширен тако што се тежина сваке мрежне везе одређује на основу њеног основног кашњења и тренутног оптерећења. На тај начин се приликом избора путање узима у обзир и променљиво стање мреже.

Коришћена је формула:

$$w(e) = d(e) \cdot \left(1 + \frac{L(e)}{100}\right)$$

којом се веће оптерећење одражава кроз повећање тежине мрежне везе. Проширени алгоритам затим користи тако израчунате тежине приликом поређења могућих путања.

Практични део рада реализован је у програмском језику Python. Направљена је графичка апликација која симулира рачунарску мрежу и омогућава избор почетног и одредишног рутера, промену оптерећења мрежних веза и израчунавање оптималне путање. Апликација приказује и укупно кашњење изабране путање, које се добија сабирањем основних кашњења свих њених веза.

Посебно је важно раздвојити тежину која се користи за избор путање од укупног кашњења које се приказује кориснику. Тежина зависи од тренутног оптерећења и служи алгоритму за избор путање, док укупно кашњење представља збир основних кашњења на изабраној путањи.

Променом оптерећења мрежних веза може доћи до промене њихових тежина, па се приликом поновног покретања алгоритма може добити другачија оптимална путања. На тај начин апликација приказује основну идеју динамичког рутирања и показује како алгоритми могу да се прилагоде променама у мрежи.

Овај пример показује како се теоријски алгоритам за проналажење најкраћег пута може применити на конкретан проблем из области рачунарских мрежа. Комбиновањем Дајкстриног алгоритма са додатним параметром који представља оптерећење мрежних веза добија се једноставан модел који омогућава проучавање утицаја променљивог стања мреже на избор путање.